MODULE 26 · DOMAIN 1 - AGENTS AND WORKFLOWS

Agent Patterns, Memory and Frameworks

The tool-use loop as the base pattern, short-term and long-term memory, the memory tool, context inside agents, and when Strands, LangGraph or PydanticAI help.

DOMAIN 1 of 8 EXAM WEIGHT 14.7% MODULE 26 of 36

admin@sthithapragna.com

A detective keeps a notebook. The case file on the desk is short-term memory. The notebook in the drawer is long-term.

admin@sthithapragna.com

The base pattern: the tool-use loop

01
Context
Instructions, the task, memory, tool definitions.

›

02
Decide
Claude answers, or asks for a tool.

›

03
Act
Your code or the harness runs the tool.

›

04
Observe
The result joins the context.

›

05
Repeat
Until done or a limit; manage context as it grows.

ON THE EXAM

Every agent, framework or SDK runs this loop. Frameworks differ in what they add around it.

Two kinds of memory

SHORT-TERM

The context window

- Everything in this conversation
- Lost when the session ends
- Managed by trimming, editing, compaction
- Module 8

LONG-TERM

Outside the window

- Files, databases, a memory tool
- Survives sessions and compaction
- Written deliberately, read when needed
- Needs rules: what, how long, who

THE TIEBREAKER

Compaction keeps context small. Memory keeps what must survive the summary. Long-running agents use both.

The memory tool

- **Client-side:** Claude requests file operations under /memories; your application runs them.
- **Six commands:** view, create, str_replace, insert, delete, rename.
- **Checked first:** the API tells Claude to view its memory directory before anything else.
- **Validate every path:** /memories/../../secrets.env is a traversal attack.
- **Pairs with compaction:** "memory preserves the information that must survive summarization".

> ON THE EXAM
>
> The tool type is memory_20250818, named memory. You decide where the files really live.

Memory with rules

- **What to keep:** decisions, commitments, preferences, progress. Not raw transcripts.
- **Scope it:** per user, per tenant, per case. Never a shared pool by accident.
- **Strip sensitive data** before writing; cap file sizes.
- **Expire it:** delete what has not been used, in line with retention policy.
- **Treat it as untrusted input** when read back: memory can be poisoned too.

> THE TAKEAWAY
>
> Memory is a database the model writes to. Give it the governance any database gets.

The frameworks the guide names

FRAMEWORK	WHAT IT IS	STRENGTH
Strands Agents	AWS open-source; "a model, tools, and a prompt"	Model-driven loop; Bedrock and Anthropic
LangGraph	"low-level orchestration framework" for stateful agents	Graphs with branches and cycles; durable state
PydanticAI	"a typed, extensible agent loop"	Typed dependencies and validated outputs
Claude Agent SDK	Claude Code's loop and tools	Built-in tools, hooks, permissions, sessions

THE TIEBREAKER

Each wraps the same tool-use loop. Choose by the control you need, not by popularity.

Raw API, Agent SDK or framework?

IF YOU NEED	LEAN TOWARDS
Full control, a small loop, nothing hidden	The raw Messages API
Files, shell, code tools, hooks, permissions	The Claude Agent SDK
Explicit branching and cycles with saved state	A graph framework such as LangGraph
Strict typed inputs and validated outputs in Python	PydanticAI
An AWS-centred model-driven agent	Strands Agents

ON THE EXAM

Anthropic: "If you do use a framework, ensure you understand the underlying code."

When a framework helps, and when it hides

HELPS

Real structure

- Branching, retries and checkpoints you would rebuild anyway
- Durable state across long runs
- Typed contracts that catch errors
- A team already fluent in it

HIDES

Extra layers

- Prompts you cannot see or change
- Abstractions over a two-call task
- Debugging through someone else's code
- Lag behind new Claude features

THE TIEBREAKER

Anthropic: frameworks "create extra layers of abstraction that can obscure the underlying prompts and responses".

Copperlet: memory for the agency-ops agent

SHORT-TERM

Inside a case

- Compaction on long investigations
- Subagent summaries, not raw logs
- Pinned case block each turn

LONG-TERM

Across cases

- Memory tool, one folder per tenancy
- Commitments, disputes, agreed plans
- Paths validated; personal data stripped
- Expires 12 months after the tenancy ends

THE TAKEAWAY

No framework added: the Agent SDK covers the loop, and LangGraph would add layers nothing requires.

> A customer success agent helps the same clients over many weeks. Each new session starts with no knowledge of earlier agreements, and clients must repeat themselves.
>
> Full transcripts are too long to reload.

What should the team add?

A. Long-term memory per client, storing agreements and preferences

B. A larger context window, reloading every past transcript

C. A longer system prompt listing all clients' agreements

D. Compaction within each session

◆ PAUSE AND DECIDE

What should the team add?

A. Long-term memory per client, storing agreements and preferences
Survives sessions; stores only what matters.

B. A larger context window, reloading every past transcript
Bloat, cost and rot.

C. A longer system prompt listing all clients' agreements
Everyone's data, in every call.

D. Compaction within each session
Helps one session, not across them.

THE SIGNAL

"Across sessions" is long-term memory. Compaction only helps within one.

> An application implements the memory tool by joining each path Claude sends onto a base directory and reading or writing the file. A tester gets the agent to read a configuration file outside the memory folder.
>
> The team wants to close the hole.

What should they do?

A Tell Claude in the system prompt to stay inside the memory folder

B Rename the memory folder to something hard to guess

C Resolve every path, rejecting any outside /memories

D Turn off the memory tool for all users

◆ PAUSE AND DECIDE

What should they do?

A Tell Claude in the system prompt to stay inside the memory folder

Guidance; the attack goes round it.

B Rename the memory folder to something hard to guess

Obscurity, not a fix.

C Resolve every path, rejecting any outside /memories

Anthropic's documented defence.

D Turn off the memory tool for all users

Loses the feature over a fixable bug.

CURRENCY CHECK

The memory tool is client-side: your handler must enforce the boundary, in code.

> A claims process has explicit branches and loops: request missing documents and wait, re-check after each upload, escalate after three failures, and resume after a server restart without losing its place.
>
> The team writes Python and wants the flow to be visible and durable.

Which approach fits best?

A. A single long prompt describing every branch

B. A fully autonomous agent with no explicit flow

C. The raw Messages API with no saved state

D. A graph framework such as LangGraph, with persisted state

◆ PAUSE AND DECIDE

Which approach fits best?

A A single long prompt describing every branch

Not durable; flow hidden in text.

B A fully autonomous agent with no explicit flow

The flow is known; do not hide it.

C The raw Messages API with no saved state

Loses its place on restart.

D A graph framework such as LangGraph, with persisted state

Explicit branches, cycles, durability.

THE SIGNAL

Explicit branches and cycles, plus durable state: a graph framework earns its place.

A team uses a heavy agent framework for a feature that makes two Claude calls in sequence. When outputs go wrong, they cannot see the exact prompts the framework sends, and debugging takes days.

No other part of the framework is used.

What should they do?

A Replace it with two direct API calls they can see and test

B Add a second framework to log the first one's prompts

C Raise the framework's verbosity setting and keep it

D Move to a larger model to reduce wrong outputs

◆ PAUSE AND DECIDE

What should they do?

A Replace it with two direct API calls they can see and test

Transparent, testable, simple.

B Add a second framework to log the first one's prompts

Another layer on a layer.

C Raise the framework's verbosity setting and keep it

Keeps the abstraction nobody needs.

D Move to a larger model to reduce wrong outputs

Hides the bug; does not find it.

THE SIGNAL

Two calls, hidden prompts, nothing else used: the framework hides more than it helps.

> A long-running research agent uses compaction. After compaction, it sometimes forgets which sources it has already ruled out, and repeats work. The team also wants findings to carry over to the next day's run.
>
> Compaction is otherwise working well.

What should the team add?

A Turn off compaction and let the context fill

B Restart the agent from scratch each hour

C Record ruled-out sources and findings in memory files

D Put all sources in the first message so they are never lost

◆ PAUSE AND DECIDE

What should the team add?

A Turn off compaction and let the context fill
Trades one problem for rot and overflow.

B Restart the agent from scratch each hour
Loses everything regularly.

C Record ruled-out sources and findings in memory files
Survives compaction and the next run.

D Put all sources in the first message so they are never lost
Bloat; compaction still summarises it.

THE SIGNAL

Anthropic: compaction keeps context small; memory preserves what must survive summarisation.

> An assistant stores long-term memory for many customers in one shared folder. A customer later sees a hint of another customer's details, and some memory files contain card numbers. Nothing is ever deleted.
>
> The team wants memory to be safe.

Which two changes fit best? Choose two.

- A Keep one shared folder, but ask Claude to ignore other customers' files
- B Scope memory per customer, enforced by the handler in code
- C Store full transcripts instead of summaries, for completeness
- D Strip sensitive data, and expire unused memory
- E Move memory into the system prompt, so it is always visible

◆ PAUSE AND DECIDE

Which two changes fit best? Choose two.

A Keep one shared folder, but ask Claude to ignore other customers' files

Guidance over a data boundary.

B Scope memory per customer, enforced by the handler in code

Closes the cross-customer leak.

C Store full transcripts instead of summaries, for completeness

More sensitive data, not less.

D Strip sensitive data, and expire unused memory

Fixes the card numbers and the hoarding.

E Move memory into the system prompt, so it is always visible

Everyone's memory, everywhere.

THE SIGNAL

Three symptoms: a shared pool, sensitive data, no expiry. Scope it; strip and expire it.

Module 26 in five lines

- **The tool-use loop** is the base of every agent, whatever wraps it.
- **Short-term memory** is context; **long-term** lives outside it and survives.
- **The memory tool** is client-side: validate paths, scope, strip, expire.
- **Frameworks:** LangGraph for graphs and state, PydanticAI for types, Strands for AWS.
- **Raw API, SDK or framework:** choose by control needed; understand the code underneath.

COMING UP

Next, Part 7: Claude Code.